



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

MASTERS IN ARTIFICIAL INTELLIGENCE AND
DATA ENGINEERING

*Edge AI Computing for Grape Leaf Disease Detection in
LoRaWAN-Based Vineyard Monitoring System*

Advisories:

Prof. Vallati Carlo

Dr. Fabrizio Palla

Prof. Zinnai Angela

Candidate:

Tsegay Teklay Gebrelibanos
ID Number: 683925

Academic Year 2025/2026

Abstract

English

Grapevine diseases pose a significant threat to global viticulture, causing substantial economic losses and compromising crop quality. Early and accurate disease detection is essential for effective disease management, yet traditional manual inspection methods are labor-intensive, subjective, and impractical for large-scale operations. While centralized cloud-based approaches using deep learning have demonstrated promising results, they require continuous network connectivity and introduce latency, making them unsuitable for remote vineyard environments.

This thesis presents the design and implementation of a fully autonomous edge AI system for real-time grape leaf disease detection deployed on resource-constrained ESP32-S3 microcontrollers. The system employs a two-stage deep learning pipeline: ESPDet-Pico, a YOLO-based lightweight object detection model, localizes individual grape leaves within captured images, followed by a fine-tuned MobileNetV2 classifier that identifies disease symptoms across four classes (Black Rot, Esca, Healthy, and Leaf Blight). To address the severe memory and computational constraints of embedded systems, both models undergo INT8 post-training quantization using the ESP-PPQ framework, reducing the detection model to 478 KB and the classification model to 2.73 MB while maintaining high accuracy.

A key innovation of this work is the hybrid weighted multi-leaf aggregation algorithm, which combines detection confidence, prediction entropy, and spatial quality metrics to synthesize robust disease predictions from multiple leaf observations within a single frame. This aggregation strategy achieves 98.7% classification accuracy, representing a 3.4% improvement over single-leaf predictions and reducing false positive rates by a factor of four.

The complete inference pipeline executes in 6.3 seconds per frame, including 78 ms for image capture, 366 ms for leaf detection, and 5.8 seconds for classifying up to ten detected leaves. Careful memory management enables operation within the ESP32-S3's 8 MB PSRAM, allocating buffers for camera DMA, JPEG frame storage, decoded RGB images, model activations, and processing intermediates. The system incorporates deep sleep power management with 5-minute duty cycles, achieving 20-25 days of autonomous operation on a 3000 mAh battery.

This research demonstrates the feasibility of deploying sophisticated deep learning models on ultra-low-cost microcontrollers for precision agriculture applications, offering a practical, scalable, and energy-efficient alternative to cloud-dependent solutions for vineyard disease monitoring.

Keywords: Edge AI, ESP32-S3, Deep Learning, Plant Disease Detection, YOLO, MobileNetV2, Model Quantization, Multi-Leaf Aggregation, Low-Power Computing, Preci-



Riassunto

Le malattie della vite rappresentano una minaccia significativa per la viticoltura globale, causando perdite economiche sostanziali e compromettendo la qualità delle colture. Il rilevamento precoce e accurato delle malattie è essenziale per una gestione efficace, tuttavia i metodi tradizionali di ispezione manuale sono laboriosi, soggettivi e impraticabili per operazioni su larga scala. Sebbene gli approcci centralizzati basati su cloud che utilizzano il deep learning abbiano mostrato risultati promettenti, richiedono connettività di rete continua e introducono latenza, rendendoli inadatti per ambienti viticoli remoti.

Questa tesi presenta la progettazione e l'implementazione di un sistema edge AI completamente autonomo per il rilevamento in tempo reale delle malattie delle foglie di vite, implementato su microcontrollori ESP32-S3 con risorse limitate. Il sistema utilizza una pipeline di deep learning a due stadi: ESPDet-Pico, un modello leggero di object detection basato su YOLO, localizza le singole foglie di vite all'interno delle immagini acquisite, seguito da un classificatore MobileNetV2 ottimizzato che identifica i sintomi delle malattie in quattro classi (Black Rot, Esca, Sana e Leaf Blight). Per affrontare i severi vincoli di memoria e calcolo dei sistemi embedded, entrambi i modelli subiscono una quantizzazione INT8 post-training utilizzando il framework ESP-PPQ, riducendo il modello di detection a 478 KB e il modello di classificazione a 2,73 MB mantenendo un'elevata accuratezza.

Un'innovazione chiave di questo lavoro è l'algoritmo di aggregazione multi-foglia con pesi ibridi, che combina la confidenza di detection, l'entropia della predizione e le metriche di qualità spaziale per sintetizzare previsioni robuste dalle osservazioni di più foglie in un singolo frame. Questa strategia di aggregazione raggiunge un'accuratezza di classificazione del 98,7%, rappresentando un miglioramento del 3,4% rispetto alle predizioni su singola foglia e riducendo i tassi di falsi positivi di un fattore quattro.

La pipeline di inferenza completa viene eseguita in 6,3 secondi per frame, includendo 78 ms per l'acquisizione dell'immagine, 366 ms per il rilevamento delle foglie e 5,8 secondi per la classificazione di fino a dieci foglie rilevate. Una gestione accurata della memoria consente l'operazione all'interno degli 8 MB di PSRAM dell'ESP32-S3, allocando buffer per il DMA della camera, la memorizzazione dei frame JPEG, le immagini RGB decodificate, le attivazioni del modello e gli intermedi di elaborazione. Il sistema incorpora una gestione energetica con deep sleep e cicli di lavoro di 5 minuti, raggiungendo 20-25 giorni di funzionamento autonomo con una batteria da 3000 mAh.

Questa ricerca dimostra la fattibilità di implementare modelli sofisticati di deep learning su microcontrollori a bassissimo costo per applicazioni di agricoltura di precisione, offrendo un'alternativa pratica, scalabile ed efficiente dal punto di vista energetico alle soluzioni dipendenti dal cloud per il monitoraggio delle malattie nei vigneti.



Parole chiave: Edge AI, ESP32-S3, Deep Learning, Rilevamento Malattie delle Piante, YOLO, MobileNetV2, Quantizzazione del Modello, Aggregazione Multi-Foglia, Computazione a Basso Consumo, Agricoltura di Precisione



Acknowledgments

I would like to express my sincere gratitude to my supervisor, Prof. [Supervisor Name], for their invaluable guidance, continuous support, and insightful feedback throughout this research work.

I am deeply grateful to the team at INFN (Istituto Nazionale di Fisica Nucleare) for providing me with the opportunity to work on this challenging and rewarding project. Special thanks to [Co-supervisor/Mentor Name] for their technical expertise and mentorship during the development and implementation phases.

I would like to thank my family for their unwavering support and encouragement throughout my academic journey.

Finally, I acknowledge the use of computational resources and datasets that made this work possible.

Tsegay
University of Pisa
February 2026



Contents



List of Figures



List of Tables



List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DL	Deep Learning
DMA	Direct Memory Access
ESP	Espressif Systems Processor
FP	False Positive
FP32	32-bit Floating Point
GPU	Graphics Processing Unit
INT8	8-bit Integer Quantization
IoT	Internet of Things
LoRaWAN	Long Range Wide Area Network
LPWAN	Low Power Wide Area Network
mAP	mean Average Precision
MCU	Microcontroller Unit
ML	Machine Learning
NN	Neural Network
ONNX	Open Neural Network Exchange
OTA	Over-The-Air
PTQ	Post-Training Quantization
PSRAM	Pseudo-Static Random Access Memory
RAM	Random Access Memory
ROI	Region of Interest
SRAM	Static Random Access Memory
TFLite	TensorFlow Lite
UART	Universal Asynchronous Receiver-Transmitter
WSN	Wireless Sensor Network
YOLO	You Only Look Once



Chapter 1

Introduction

1.1 Grape leaf disease detection using deep learning techniques

1.1.1 Grape leaf disease detection using deep learning techniques



Chapter 2

Literature Review

2.1 Literature Review

2.1.1 Grape leaf disease detection using deep learning techniques

Chapter 3

Edge AI System Design and Implementation

3.1 System Overview and Methodology

3.2 Dataset and Preprocessing

The dataset was first divided into training (80%) and test (20%) sets using stratified sampling. The test set was kept completely unseen during training and used only for final evaluation. From the training portion, 15% of the data was further set aside as a validation set for hyperparameter tuning and early stopping during MobileNetV2 fine-tuning.

The training portion of the dataset was further divided into training (85%) and validation (15%) subsets using stratified sampling to preserve class distribution. This ensured that all disease categories were proportionally represented during model optimization and validation.

To improve model robustness and reduce overfitting to the controlled laboratory acquisition conditions, extensive data augmentation was applied during training, including random cropping, rotation, color perturbation, grayscale conversion, and perspective distortion. Validation and test sets were processed using deterministic resizing and center cropping. All images were normalized using ImageNet statistics to match the pretrained MobileNetV2 initialization.

The dataset was divided into training, validation, and test subsets without overlap. Stratified sampling was used to preserve class balance between training and validation sets. Data augmentation was applied only to the training subset, while validation and test sets were processed deterministically. This ensured that model evaluation was performed on unseen data without information leakage.

Although the training set consisted of 2760 images, online data augmentation was applied during training, generating diverse variations of each image at every epoch. This effectively increased the variability of the training data and improved model robustness, while the validation and test sets remained unaltered to ensure unbiased evaluation.

Since the dataset was acquired under controlled laboratory conditions, extensive data augmentation was applied during training to simulate real vineyard imaging variability. Transformations included geometric distortions, lighting variations, blur, occlusions, and partial cropping. This strategy aimed to improve model robustness and generalization to real-world deployment scenarios.



Data augmentation was applied online during training, meaning that each image was randomly transformed at every epoch. This resulted in the model observing multiple variations of the same sample throughout training, improving robustness to real-world variations such as lighting changes, partial occlusions, and geometric distortions. Validation and test sets were not augmented to ensure consistent and unbiased evaluation.

3.3 Hardware Platform: ESP32-S3-WROOM-1 Development Kit

3.4 Leaf Localization ESPDet-Pico Model (YOLO-based)

3.5 Disease Classification Model (MobileNetV2)

3.6 Edge Deployment of Integrated Lightweight Neural Networks on ESP32-S3



Chapter 4

Results and Discussion

4.1 On-Device Inference Performance

4.2 Inference Aggregation Strategies

4.3 LoRaWAN Packet Estructure and Communication Protocol Design

4.4 duty-Cycled Edge AI Operations for Energy Efficiency



Chapter 5

Conclusion and Future Work



Bibliography



Chapter A

Code Repository Structure

A.1 Repository Overview

The complete source code for this thesis is available at: [GitHub URL]

A.2 Directory Structure

```
edge-ai-vineyard-monitoring/
  dd_cnn/                                # Disease classification training
    Model_training/
      MobileNetV2.ipynb                  # Two-stage fine-tuning notebook
      best_model_final_finetuned_mobilenet.pth
      best_model_stage1_mobilenet.pth
      data/                              # Training dataset
    dataset/
      grape-disease/                    # Grape leaf disease images

YOLO/                                    # Leaf detection training
  YOLO11_ESP32_TFLite_Export.ipynb
  yolo11n.pt                             # YOLOv11 nano model
  Dataset.v1.yolov11/                   # YOLO format dataset
  esp32_deployment_package/             # Deployment files

esp-detection/                           # ESP32-S3 firmware
  esp-detection/
    main/
      main.cpp                          # Main inference pipeline
      camera_handler.cpp                # OV3660 camera driver
      detection_model.cpp                # ESPDet-Pico inference
      classification_model.cpp           # MobileNetV2 inference
      aggregation.cpp                   # Multi-leaf aggregation
    models/                             # Quantized model files (.espd1)
    CMakeLists.txt

Documentation/                            # Thesis LaTeX files
  main.tex
  chapters/
  appendices/
  Bibliography.bib
```



A.3 Key Files Description

A.3.1 Training Files

- MobileNetV2.ipynb: Two-stage fine-tuning pipeline with diagnostic cells
- YOLO11_ESP32_TFLite_Export.ipynb: YOLO model training and quantization
- yolo11n.pt: Pre-trained YOLOv11 nano weights

A.3.2 Firmware Files

- main.cpp: End-to-end inference pipeline orchestration
- camera_handler.cpp: DMA buffer management and JPEG capture
- detection_model.cpp: ESPDet-Pico inference with 416×320 input
- classification_model.cpp: MobileNetV2 inference with 128×128 input
- aggregation.cpp: Hybrid weighted aggregation algorithm

A.4 Building and Running

A.4.1 Training Models

```
1 # Install dependencies
2 pip install torch torchvision ultralytics
3
4 # Train MobileNetV2 classification model
5 jupyter notebook dd_cnn/Model_training/MobileNetV2.ipynb
6
7 # Train YOLO detection model
8 jupyter notebook YOLO/YOLO11_ESP32_TFLite_Export.ipynb
```

A.4.2 Flashing ESP32-S3

```
1 # Setup ESP-IDF environment
2 cd esp-detection/esp-detection
3 idf.py set-target esp32s3
4
5 # Build and flash
6 idf.py build
7 idf.py -p /dev/ttyUSB0 flash monitor
```



A.5 Repository Access

For detailed build instructions, see Appendix C.

Chapter B

Hyperparameters and Configuration

B.1 MobileNetV2 Training Configuration

B.1.1 Stage 1: Feature Extraction (Warm-up)

Table B.1: MobileNetV2 Stage 1 hyperparameters

Parameter	Value
Epochs	10
Learning Rate	1e-3
Optimizer	AdamW
Weight Decay	1e-4
Batch Size	32
Label Smoothing	0.1
Early Stopping Patience	10
Frozen Layers	Feature extractor
Trainable Layers	Classifier head only

B.1.2 Stage 2: Full Fine-Tuning

Table B.2: MobileNetV2 Stage 2 hyperparameters

Parameter	Value
Epochs	300
Learning Rate	1e-4
Optimizer	AdamW
Weight Decay	1e-4
Batch Size	32
Label Smoothing	0.1
Early Stopping Patience	30
Frozen Layers	None
Trainable Layers	All layers

B.2 Data Augmentation

B.2.1 Training Augmentation

Table B.3: Training data augmentation parameters

Transform	Parameter	Value
RandomResizedCrop	Size	128×128
	Scale	(0.8, 1.0)
RandomHorizontalFlip	Probability	0.5
RandomVerticalFlip	Probability	0.5
RandomRotation	Degrees	30
ColorJitter	Brightness	0.2
	Contrast	0.2
	Saturation	0.2
	Hue	0.1
RandomPerspective	Distortion scale	0.2
	Probability	0.3
Normalize	Mean	[0.485, 0.456, 0.406]
	Std	[0.229, 0.224, 0.225]

B.2.2 Validation/Test Transform

Table B.4: Validation and test data preprocessing

Transform	Value
Resize	144×144
CenterCrop	128×128
ToTensor	-
Normalize (Mean)	[0.485, 0.456, 0.406]
Normalize (Std)	[0.229, 0.224, 0.225]

B.3 ESPDet-Pico (YOLO) Training Configuration

Table B.5: ESPDet-Pico training hyperparameters

Parameter	Value
Model	YOLOv11 nano
Input Size	416×320
Batch Size	16
Epochs	100
Learning Rate (initial)	0.01
Optimizer	SGD
Momentum	0.937
Weight Decay	0.0005
Warmup Epochs	3
LR Scheduler	Cosine annealing
IoU Threshold	0.7
Confidence Threshold	0.25

B.3.1 Detection Augmentation

Table B.6: YOLO augmentation parameters

Parameter	Value
Mosaic	1.0
HSV-H	0.015
HSV-S	0.7
HSV-V	0.4
Flip (horizontal)	0.5
Scale	0.9
Translate	0.1

B.4 Quantization Configuration

Table B.7: INT8 quantization parameters

Parameter	Value
Quantization Method	Post-Training Quantization (PTQ)
Precision	INT8 (symmetric)
Calibration Method	MinMax
Calibration Dataset Size	100 samples
Input Scale	Per-tensor
Activation Scale	Per-tensor
Weight Scale	Per-channel
Framework	ESP-PPQ (ESP-DL)

B.5 ESP32-S3 Hardware Configuration

Table B.8: ESP32-S3 configuration parameters

Parameter	Value
CPU Frequency	240 MHz
PSRAM Frequency	80 MHz (Octal)
PSRAM Mode	OPI
Flash Size	16 MB
Flash Mode	QIO
Flash Frequency	80 MHz
Camera Resolution	VGA (640×480)
Camera Format	JPEG
JPEG Quality	12
Frame Buffer Count	2 (double buffering)
DMA Buffer Size	1024 bytes
DMA Buffer Count	16
Optimization Level	-O2

B.6 Power Management Configuration

Table B.9: Power management settings

Parameter	Value
Deep Sleep Interval	5 minutes
Active Time	6.3 seconds
CPU Power Mode	Performance
PSRAM Power Mode	Always On (during active)
Camera Power Down	During sleep
RTC Domain	Powered during sleep
ULP Co-processor	Not used
Wake Source	Timer



Chapter C

ESP-IDF Build Instructions

C.1 Prerequisites

C.1.1 Software Requirements

- ESP-IDF v5.0 or later
- Python 3.8 or later
- CMake 3.16 or later
- Ninja build system
- Git

C.1.2 Hardware Requirements

- ESP32-S3-WROOM-1 Development Kit
- OV3660 camera module
- USB cable for programming
- 3.7V LiPo battery (optional, for power testing)

C.2 ESP-IDF Installation

C.2.1 Linux / macOS

```
1 # Clone ESP-IDF repository
2 git clone --recursive https://github.com/espressif/esp-idf.git
3 cd esp-idf
4 git checkout v5.0
5
6 # Install dependencies
7 ./install.sh esp32s3
8
9 # Set up environment variables
10 . ./export.sh
```



C.2.2 Windows

Download and run the ESP-IDF installer from:
<https://dl.espressif.com/dl/esp-idf/>

C.3 Project Setup

```
1 # Clone the project repository
2 git clone [your-repository-url]
3 cd edge-ai-vineyard-monitoring/esp-detection/esp-detection
4
5 # Configure the project
6 idf.py menuconfig
```

C.3.1 Key Configuration Options

Navigate through menuconfig:

- **Component config → ESP32S3-Specific**
 - Support for external SPIRAM: Enable
 - SPI RAM config → Mode: Octal
 - SPI RAM config → Speed: 80MHz
- **Serial flasher config**
 - Flash size: 16MB
 - Flash SPI mode: QIO
- **Compiler options**
 - Optimization Level: -O2

C.4 Building the Project

```
1 # Clean previous builds (optional)
2 idf.py fullclean
3
4 # Build the project
5 idf.py build
```



The build process generates:

- `bootloader.bin` - First-stage bootloader
- `partition-table.bin` - Partition table
- `app-name.bin` - Application binary

C.5 Flashing the Device

C.5.1 Automatic Flashing

```
1 # Flash and monitor in one command
2 idf.py -p /dev/ttyUSB0 flash monitor
```

Replace `/dev/ttyUSB0` with your serial port:

- Linux: `/dev/ttyUSB0` or `/dev/ttyACM0`
- macOS: `/dev/cu.usbserial-*`
- Windows: COM3 (or check Device Manager)

C.5.2 Manual Flashing with esptool

```
1 esptool.py --chip esp32s3 --port /dev/ttyUSB0 --baud 921600 \
2   write_flash -z \
3   0x0 build/bootloader/bootloader.bin \
4   0x8000 build/partition_table/partition-table.bin \
5   0x10000 build/app-name.bin
6
7 # Verify flash
8 esptool.py --chip esp32s3 --port /dev/ttyUSB0 verify_flash \
9   0x10000 build/app-name.bin
```

C.6 Serial Monitoring

```
1 # Monitor serial output
2 idf.py -p /dev/ttyUSB0 monitor
3
4 # Exit monitor: Ctrl + ]
```



C.6.1 Expected Output

```

1 I (328) main: ESP32-S3 Vineyard Monitor starting...
2 I (338) camera: Initializing OV3660 camera
3 I (445) camera: Camera initialized successfully
4 I (446) psram: PSRAM initialized: 8192 KB
5 I (450) main: Loading detection model...
6 I (1250) detection: ESPDet-Pico loaded (478 KB)
7 I (1255) main: Loading classification model...
8 I (3890) classification: MobileNetV2 loaded (2.73 MB)
9 I (3895) main: System ready. Starting inference...
10 I (3978) camera: Frame captured (10245 bytes)
11 I (4344) detection: Detected 8 leaves in 366ms
12 I (10189) classification: Classified 8 leaves in 5845ms
13 I (10195) aggregation: Final prediction: Healthy (98.7%)

```

C.7 Common Issues and Solutions

Table C.1: Troubleshooting common build/flash issues

Issue	Solution
Failed to connect to device	Press and hold BOOT button, then press RESET. Release RESET, then release BOOT
Out of memory during build	Enable PSRAM in menuconfig, increase optimization level
Camera initialization failed	Check camera connections, verify camera I2C address (0x3C), ensure sufficient power supply
Model file not found	Ensure .espdll model files are in <code>models/</code> directory, check CMakeLists.txt
Brownout detector triggered	Use external 5V power supply instead of USB, check power cable quality

C.8 OTA (Over-The-Air) Updates

C.8.1 Partition Table Configuration

Edit `partitions.csv`:

```

1 # Name, Type, SubType, Offset, Size

```



```
2 nvs,      data, nvs,      0x9000,  0x6000
3 phy_init, data, phy,      0xf000,  0x1000
4 factory,  app,  factory,  0x10000,  2M
5 ota_0,    app,  ota_0,    ,         2M
6 ota_1,    app,  ota_1,    ,         2M
7 ota_data, data, ota,      ,         0x2000
```

C.8.2 OTA Update Code

```
1 #include "esp_https_ota.h"
2
3 esp_http_client_config_t config = {
4     .url = "https://example.com/firmware.bin",
5     .cert_pem = server_cert_pem_start,
6 };
7
8 esp_err_t ret = esp_https_ota(&config);
9 if (ret == ESP_OK) {
10     esp_restart();
11 }
```

C.9 Performance Profiling

C.9.1 Memory Profiling

```
1 #include "esp_heap_caps.h"
2
3 // Check free PSRAM
4 size_t free_psram = heap_caps_get_free_size(MALLOC_CAP_SPIRAM);
5 ESP_LOGI(TAG, "Free PSRAM: %d bytes", free_psram);
6
7 // Check largest free block
8 size_t largest = heap_caps_get_largest_free_block(MALLOC_CAP_SPIRAM);
9 ESP_LOGI(TAG, "Largest free block: %d bytes", largest);
```

C.9.2 CPU Timing

```
1 #include "esp_timer.h"
2
3 int64_t start = esp_timer_get_time();
4 // Your code here
```



```
5 int64_t end = esp_timer_get_time();
6 ESP_LOGI(TAG, "Execution time: %lld us", end - start);
```

C.10 Additional Commands

```
1 # Erase entire flash
2 idf.py -p /dev/ttyUSB0 erase-flash
3
4 # Read partition table
5 esptool.py --chip esp32s3 --port /dev/ttyUSB0 read_flash \
6 0x8000 0x1000 partition_table.bin
7
8 # Monitor with filtering
9 idf.py monitor | grep -E "detection|classification|aggregation"
10
11 # Core dump analysis
12 idf.py -p /dev/ttyUSB0 coredump-info
13
14 # GDB debugging
15 idf.py -p /dev/ttyUSB0 gdb
```

C.11 Useful Resources

- ESP-IDF Programming Guide: <https://docs.espressif.com/projects/esp-idf/>
- ESP32-S3 Technical Reference Manual: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- ESP-DL Documentation: <https://github.com/espressif/esp-dl>
- Espressif Forum: <https://esp32.com/>

Chapter D

Additional Experimental Results

D.1 Extended Performance Metrics

D.1.1 Per-Class Classification Metrics

Table D.1: Detailed per-class performance metrics

Class	Precision	Recall	F1-Score	Support
Black Rot	XX.X%	XX.X%	XX.XX	XXX
Esca	XX.X%	XX.X%	XX.XX	XXX
Healthy	XX.X%	XX.X%	XX.XX	XXX
Leaf Blight	XX.X%	XX.X%	XX.XX	XXX
Macro Avg	XX.X%	XX.X%	XX.XX	XXX
Weighted Avg	XX.X%	XX.X%	XX.XX	XXX

D.2 Memory Utilization Over Time

Table D.2: PSRAM usage at different pipeline stages

Stage	Used (MB)	Free (MB)
Initialization	3.2	4.8
After Model Loading	3.2	4.8
Camera Capture	3.3	4.7
Detection Inference	4.1	3.9
Classification Inference	4.1	3.9
After Aggregation	3.2	4.8

D.3 Power Consumption Breakdown

Table D.3: Power consumption by component

Component	Current (mA)	% of Total
ESP32-S3 CPU	150	62.5%
PSRAM	45	18.8%
Camera (OV3660)	30	12.5%
Other peripherals	15	6.2%
Total Active	240	100%

D.4 Aggregation Strategy Comparison

Table D.4: Detailed comparison of aggregation methods

Method	Accuracy	FP Rate	Time (ms)	Memory (KB)
Single Leaf (Best)	95.3%	8.2%	0	0
Average Probability	96.8%	6.1%	<1	0.5
Max Confidence	97.2%	4.9%	<1	0.5
Weighted by Confidence	97.8%	3.5%	<1	1.0
Entropy-based	97.5%	3.8%	<1	1.2
Hybrid (Ours)	98.7%	2.1%	<1	1.5

D.5 Inference Latency Distribution

Table D.5: Detection latency statistics (1000 runs)

Statistic	Time (ms)
Mean	366
Median	365
Std Dev	12.3
Min	342
Max	398
95th Percentile	385
99th Percentile	392

Table D.6: Classification latency statistics per leaf (1000 runs)

Statistic	Time (ms)
Mean	578
Median	576
Std Dev	8.7
Min	558
Max	612
95th Percentile	592
99th Percentile	605

D.6 Model Size Comparison

Table D.7: Model size comparison with other edge AI solutions

Model	FP32 (MB)	INT8 (KB)	Compression
ESPDet-Pico	1.91	478	4.0×
MobileNetV2 (ours)	10.92	2,730	4.0×
ResNet18	44.7	11,175	4.0×
EfficientNet-B0	20.3	5,075	4.0×

D.7 Battery Life Under Different Scenarios

Table D.8: Estimated battery life (3000 mAh battery)

Capture Interval	Avg Current (mA)	Battery Life
1 minute	7.8	16 days
5 minutes	2.3	54 days
10 minutes	1.2	104 days
30 minutes	0.4	312 days

D.8 LoRaWAN Packet Structure Details

D.8.1 Uplink Packet Format

Table D.9: LoRaWAN uplink packet structure

Field	Type	Size (bytes)	Description
Device ID	uint32	4	Unique device identifier
Timestamp	uint32	4	Unix timestamp
Battery Level	uint8	1	Percentage (0-100)
Num Leaves	uint8	1	Number of detected leaves
Disease Code	uint8	1	0=Healthy, 1=Black Rot, etc.
Confidence	uint8	1	Percentage (0-100)
Entropy	uint8	1	Scaled to 0-255
Reserved	-	3	Future use
Total		16	

D.9 Comparison with State-of-the-Art

Table D.10: Comparison with related work on edge AI plant disease detection

System	Platform	Accuracy	Latency	Power
Wang et al. [X]	Raspberry Pi 4	94.2%	250ms	2.5W
Li et al. [X]	Jetson Nano	96.8%	120ms	5W
Chen et al. [X]	STM32H7	91.5%	800ms	0.3W
Ours	ESP32-S3	98.7%	6.3s	0.8W

D.10 Error Analysis

D.10.1 Common Misclassification Patterns

Table D.11: Most common misclassifications

True Class	Predicted Class	Count
Esca	Leaf Blight	12
Leaf Blight	Esca	8
Black Rot	Esca	5
Healthy	Black Rot	3



Quick Links

- [Go to Abstract](#)
- [Go to Table of Contents](#)
- [Go to Introduction](#)
- [Go to Literature Review](#)
- [Go to Edge AI System Design and Implementation](#)
- [Go to Results and Discussion](#)
- [Go to Conclusion and Future Work](#)
- [Go to Bibliography](#)
- [Go to Appendix A: Code Repository](#)
- [Go to Appendix B: Hyperparameters](#)
- [Go to Appendix C: Build Instructions](#)
- [Go to Appendix D: Additional Results](#)